

Implementasi CNN Berbasis VGG16 untuk Klasifikasi Citra Buah Tomat dan Apel



Dosen Pengampuh:

Sutarno, M.T.

Anggun Islami, M.Kom

Disusun Oleh:

Nama : Dzhahira Shafa

NIM : 09011382227150

Kelas : SKU6

JURUSAN SISTEM KOMPUTER

FAKULTAS ILMU KOMPUTER

UNIVERSITAS SRIWIJAYA PALEMBANG

1. Pendahuluan

Dalam era digital saat ini, teknologi pengolahan citra digital telah menjadi salah satu bidang yang berkembang pesat dan memiliki berbagai penerapan dalam kehidupan sehari-hari, seperti dalam bidang kesehatan, keamanan, industri, dan sebagainya. Salah satu cabang dari pengolahan citra digital yang sedang banyak dikembangkan adalah klasifikasi citra menggunakan metode deep learning.

Deep learning merupakan bagian dari machine learning yang menggunakan arsitektur jaringan saraf tiruan (neural network) dengan banyak lapisan untuk mengekstraksi fitur dari data secara otomatis. Teknik ini telah terbukti sangat efektif dalam pengenalan pola, termasuk dalam klasifikasi citra.

Tujuan dari tugas ini adalah membangun model klasifikasi citra dengan memanfaatkan jaringan saraf konvolusional (CNN) serta menerapkan berbagai teknik seperti augmentasi data, transfer learning, dan evaluasi performa model. Proyek ini juga bertujuan untuk memberikan pemahaman praktis dalam penerapan deep learning untuk klasifikasi citra.

2. Metode dan Bahan

2.1 Pengumpulan & Persiapan Dataset

Data set yang digunakan berasal dari Kaggle. Dengan jumlah data yang digunakan sebanyak 391. Dataset yang digunakan terdiri dari dua kelas, yang disusun dalam struktur direktori terpisah. Dataset ini terbagi dalam direktori train dan test, data train sebanyak 294 dan data test sebanyak 97. Label citra otomatis dikenali berdasarkan nama folder (tomat dan apel).

2.2 Pra-pemrosesan Citra

Sebelum gambar digunakan dalam proses pelatihan model, terlebih dahulu dilakukan tahap preprocessing untuk menyesuaikan data dengan format yang dapat diproses oleh arsitektur CNN, khususnya VGG16.

- **Label Data**
Labeling Data dilakukan otomatis berdasarkan direktori. Pada data ini dimana tomatoes itu untuk gambar tomat dan apples untuk gambar apel
- **Augmentasi Data**

Augmentasi pada Data menggunakan ImageDataGenerator dari Keras, dilakukan augmentasi berupa rescale, zoom, dan validation split. Resize & Normalisasi semua citra diubah ke ukuran 224x224 piksel dan dinormalisasi ke skala [0, 1]. Resize 224x224 piksel,

sesuai dengan input standar untuk arsitektur CNN seperti VGG16 atau ResNet.

```
[4] batch_size = 32
img_size = 224

datagen = ImageDataGenerator(rescale=1/255.,
                             zoom_range=0.8,
                             validation_split=0.15)

train_generator = datagen.flow_from_directory('/content/drive/MyDrive/UAS Citra/train',
                                              target_size=(img_size, img_size),
                                              batch_size=batch_size,
                                              shuffle=True,
                                              class_mode='categorical',
                                              subset='training')

validation_generator = datagen.flow_from_directory('/content/drive/MyDrive/UAS Citra/test',
                                                  target_size=(img_size, img_size),
                                                  batch_size=batch_size,
                                                  shuffle=False,
                                                  class_mode='categorical',
                                                  subset='validation')
```

Found 251 images belonging to 2 classes.
Found 14 images belonging to 2 classes.

2.3 Pembuatan model CNN

Model CNN dibangun dengan pendekatan transfer learning dari model VGG16. Menggunakan VGG16 tanpa fully connected layer (include_top=False). Kemudian menambahkan layer pooling dan dense (512 unit, ReLU). Output layer terdiri dari 2 neuron (softmax) untuk klasifikasi biner.

```
[4] base_model = VGG16(include_top=False,
                       weights='imagenet',
                       input_shape=(img_size, img_size, 3))

last = base_model.layers[-2].output
x = GlobalAveragePooling2D()(last)
x = Dense(512, 'relu')(x)
x = Dense(2, activation='softmax')(x)
model = Model(inputs=base_model.input, outputs=x)
```

2.4 Kompilasi dan Pelatihan Model

Model dikompilasi menggunakan Optimizer Adam dengan learning rate = 0.0001. Loss Function menggunakan Categorical Crossentropy untuk klasifikasi multi class. Metrics untuk mengukur performa model berdasarkan dengan Accuracy.

```
model.compile(loss='categorical_crossentropy',
              optimizer=Adam(learning_rate=0.0001),
              metrics=['accuracy'])

model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808

block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense (Dense)	(None, 512)	262,656
dense_1 (Dense)	(None, 2)	1,026

Total params: 14,978,370 (57.14 MB)
Trainable params: 14,978,370 (57.14 MB)
Non-trainable params: 0 (0.00 B)

Selama pelatihan, digunakan ModelCheckpoint, EarlyStopping, dan ReduceLROnPlateau untuk menghindari overfitting dan mempercepat konvergensi. Data pelatihan dan validasi diproses secara real-time menggunakan generator dengan melatih model maksimal 10 epoch.

```

model_name = "model.keras" # Change the extension to .keras
checkpoint = ModelCheckpoint(model_name,
                             monitor="val_loss",
                             mode="min",
                             save_best_only = True,
                             verbose=1)

earlystopping = EarlyStopping(monitor='val_loss',min_delta = 0, patience = 5, verbose = 1, restore_best_weights=True)

learning_rate_reduction = ReduceLROnPlateau(monitor='val_loss',
                                             patience=3,
                                             verbose=1,
                                             factor=0.2,
                                             min_lr=0.00000001)

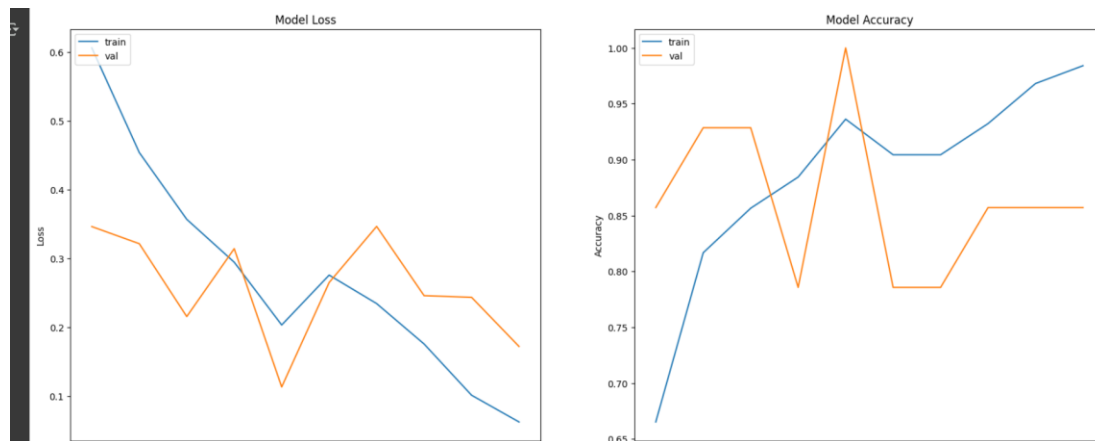
[6] history = model.fit(train_generator,
                       epochs=10,
                       validation_data=validation_generator,
                       callbacks=[checkpoint,earlystopping,learning_rate_reduction])

/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `self.warn_if_super_not_called()`

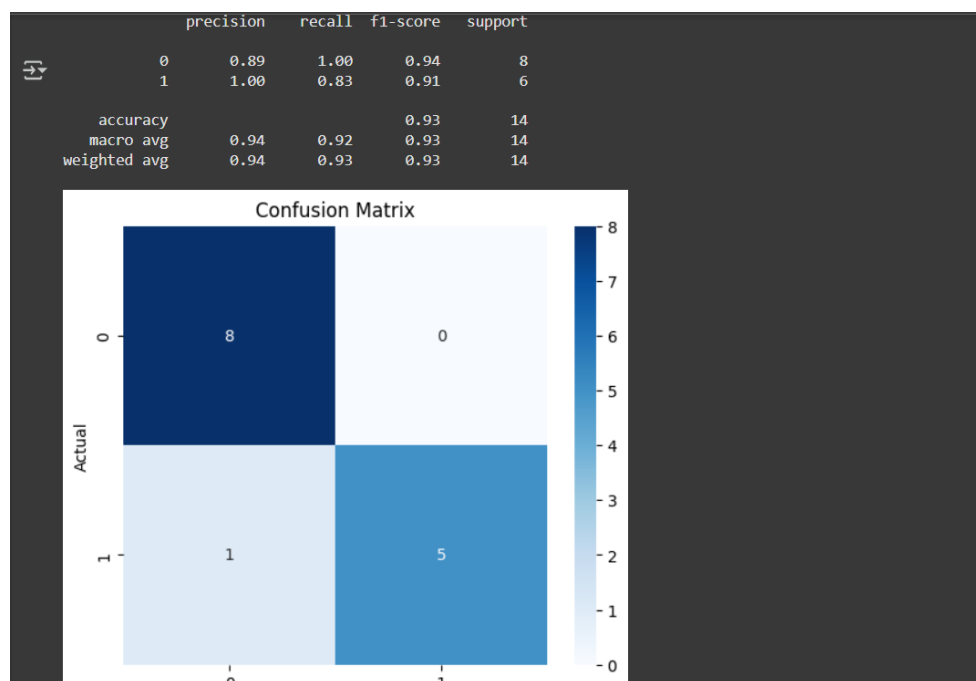
```

3. Hasil dan Pembahasan Evaluasi Model

Hasil dari proses training atau pelatihan dalam CNN dengan arsitektur VGG16 adalah terbentuknya sebuah model. Model menunjukkan performa pelatihan yang baik, dengan Loss training yang menurun konsisten dan akurasi training yang tinggi. Pada validasi menunjukkan fluktuasi wajar, namun akurasi tetap tinggi, mengindikasikan model memiliki kemampuan generalisasi yang baik. Tidak terjadi overfitting parah, karena tidak ada gap ekstrem antara training dan validation. Dapat dilihat dari gambar dibawah ini.



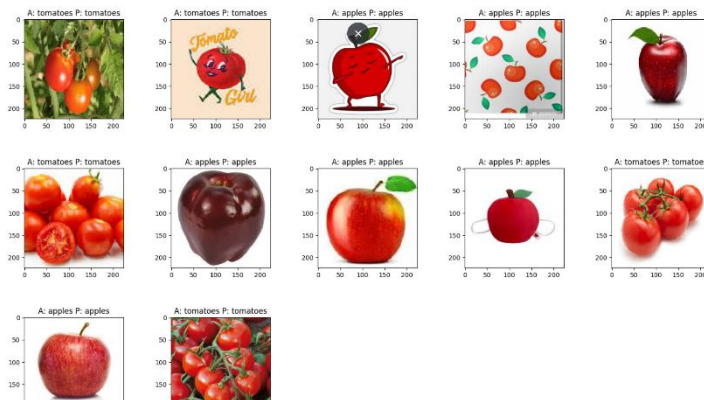
Model dievaluasi menggunakan metrik dan mendapatkan hasil evaluasi menunjukkan akurasi mencapai 93%. Classification Report pada library sklearn akan digunakan untuk melihat nilai akurasi, presisi, recal dan f1-score. Hasil dari evaluasi dapat dilihat pada gambar dibawah ini.



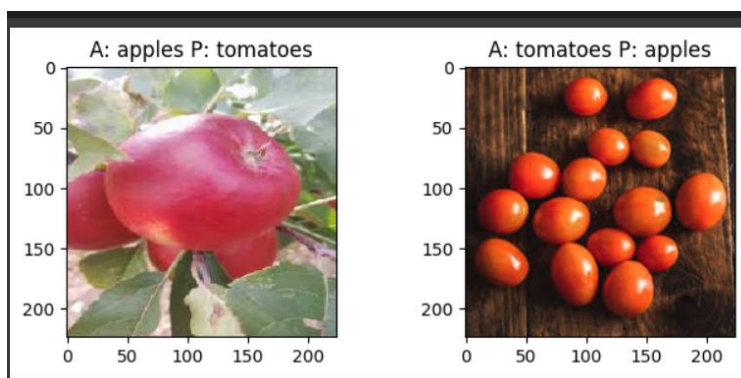
4. Menampilkan Gambar yang di prediksi benar dan salah

	filename	actual	predicted	Same
0	apples/img_p1_111.jpeg	apples	tomatoes	False
1	apples/img_p1_121.jpeg	apples	apples	True
2	apples/img_p1_127.jpeg	apples	apples	True
3	apples/img_p1_131.jpeg	apples	apples	True
4	apples/img_p1_133.jpeg	apples	apples	True
5	apples/img_p1_139.jpeg	apples	apples	True
6	apples/img_p1_14.jpeg	apples	apples	True
7	apples/img_p1_143.jpeg	apples	apples	True
8	tomatoes/img_p1_105.jpeg	tomatoes	apples	False
9	tomatoes/img_p1_107.jpeg	tomatoes	tomatoes	True

Prediksi Benar



Prediksi Salah



Kesimpulan

Berdasarkan hasil eksperimen yang dilakukan, model CNN berbasis arsitektur VGG16 mampu mengklasifikasikan citra apel dan tomat dengan akurasi mencapai 93%. Dari tugas ini dapat disimpulkan bahwa penggunaan CNN berbasis VGG16 sangat efektif dalam klasifikasi citra biner. Kombinasi augmentasi data, optimasi hyperparameter, dan evaluasi menyeluruh memberikan performa model yang memuaskan.